

Procédure de test — Chaîne Catalogue → UO (projet Train System)

Objet : valider la chaîne complète d'instanciation et de synchronisation des UO : génération du catalogue depuis le Word, création d'une UO en une commande, saisie ingénieur, synchronisation, KPI — puis robustesse du moteur face à des modifications du fichier (colonnes ajoutées, Manifeste modifié, tableau renommé). **Durée estimée** : 60-90 minutes. **Tous les résultats attendus de cette procédure ont été vérifiés le 2026-06-12.**

Note ce que tu obtiens à chaque étape dans le classeur de suivi `Validation-UO-TrainSystem.xlsx` (statut OK/KO + remarques). En cas de KO, continue si possible — le bilan global compte plus qu'un blocage isolé.

Règles de bonne conduite

Ces règles évitent les faux KO et les situations de debug inutiles.

Avant chaque sync : - **Ferme toujours le fichier Excel** avant de lancer une commande — sinon le moteur obtient un fichier verrouillé et la sync échoue avec une erreur trompeuse. - **Rouvre le fichier après** pour constater le résultat — les valeurs BIND ne s'actualisent qu'à l'ouverture.

Sur les tableaux Excel : - **Ne jamais renommer un tableau** (ex. `tbl_act`, `tbl_oil`) — les noms sont référencés mot pour mot dans le `_Manifeste`. Un tableau renommé disparaît silencieusement du store sans message d'erreur (le moteur retourne `[]`). - **Tu peux ajouter des colonnes** librement — le moteur les intègre automatiquement. Les receveurs qui ne les demandent pas les ignorent. - **Tu peux ajouter des feuilles** — elles sont ignorées si le Manifeste n'y fait pas référence.

Sur le `_Manifeste` : - **Toute modification prend effet immédiatement** à la prochaine sync — pas besoin de recréer l'UO. - **Ne change pas le FILE_ID** en cours de test — ça crée une nouvelle clé dans le store, l'ancienne reste orpheline, et les valeurs semblent doubler. - **Une faute de syntaxe MXL** dans le Manifeste bloque la sync entière avec une erreur de parsing explicite — corrige et relance.

Sur le store : - Utilise `python -m src store clear` pour repartir proprement entre deux scénarios distincts. Inutile entre deux syncs du même fichier. - `python -m src status` à tout moment pour voir ce qui est réellement dans le store.

Prérequis

1. Dépôt à jour : `git pull` dans le dossier `SysEng`.

2. Dépendances : `pip install -r requirements.txt` (une fois).
3. **Le fichier Word UO TS.docx** — il est **confidentiel** et n'est PAS sur GitHub : il doit t'être transmis par canal interne (mail/Teams). Note le chemin où tu l'enregistres.
4. Toutes les commandes se lancent **depuis la racine du dossier SysEng**.
5. Règle d'or : **toujours fermer le fichier Excel avant de lancer une commande**, et le rouvrir après pour constater le résultat.

Étape 1 — Générer le catalogue depuis le Word

```
python projet_TrainSystem/parse_catalogue.py "CHEMIN\VERS\UO TS.docx"
python projet_TrainSystem/build_catalogue.py
```

Attendu : - La 1^{re} commande affiche un tableau de **10 lignes** (L09U1 → L13U2) avec les comptes d'activités par UO.
 - La 2^e affiche : [OK] Catalogue_UO_TrainSystem.xlsx – 10 UO types, 54 activités, 97 livrables, 88 données d'entrée.
 - Ouvre projet_TrainSystem\Catalogue_UO_TrainSystem.xlsx : 4 onglets (Index, Catalogue_Activites, Catalogue_Livrables, Catalogue_DonneesEntree). Dans Catalogue_Activites, filtre la colonne `uo_type` sur L09U1 : **13 lignes**.

Étape 2 — Tests négatifs (codes invalides)

```
python projet_TrainSystem/creer_uo.py CODE-INVALIDE
python projet_TrainSystem/creer_uo.py L99U9-TEST
```

Attendu : deux messages d'erreur clairs (pas de plantage Python) : - [ERR] Code invalide 'CODE-INVALIDE' – attendu : L09U1-PROJET-SYSTEME - [ERR] UO type 'L99U9' absent du catalogue. Disponibles : L09U1, L09U2, ...

Étape 3 — Créer l'UO de test

```
python projet_TrainSystem/creer_uo.py L09U1-TEST01-CLIM --se "Ton Nom" --heures 240 --projet "Projet Test"
```

Attendu : [OK] ...L09U1-TEST01-CLIM.xlsx créé. Ouvre-le et vérifie : - **11 feuilles** : General, Description_Besoin, Donnees_Entree, Activites, Livrables, OIL, KPI, Dashboard, Planning, Orga, _Manifeste ; - General : projet, système, ton nom, **240** en Heures vendues ; - Activites : **13 lignes** pré-remplies (applicable=OUI, avancement=0, heures_allouees calculées ≈ 18,46 h chacune) ; - Description_Besoin : le cahier des charges complet du type L09U1 ; - Livrables : 21 lignes · Donnees_Entree : 21 lignes ; - OIL : 1 ligne d'exemple PO-000 ; - Les listes déroulantes fonctionnent (colonne statut d'Activites, etc.).

Ferme le fichier.

Étape 4 — Synchronisation à blanc (UO vierge)

```
python -m src store clear
python scripts/valider_un.py projet_TrainSystem/L09U1-TEST01-CLIM.xlsx
```

Attendu (console) : PUSH = 8, BIND = 7, [OK] Execution terminée sans erreur. — et 8 clés uo.L09U1-TEST01-CLIM.* listées dans le store. Rouvre le fichier : onglet KPI → Avancement 0 ; Points ouverts 0 ; Points fermés 1 (la ligne d'exemple) ; Santé **VERT**. Referme.

Étape 5 — Saisie ingénieur (scénario imposé)

Ouvre le fichier et saisis **exactement** ceci :

Feuille Activités (les lignes du tableau, ligne 2 = première activité) : | Ligne | applicable | statut | avancement | heures_consommees | |---|---|---|---|---| | 2 | OUI | EN_COURS | **50** | **10** | | 3 | OUI | TERMINEE | **100** | **20** | | 5 | **NON** | — | — | — | | toutes les autres | OUI (inchangé) | inchangé | 0 | 0 |

Feuille OIL — remplace la ligne d'exemple et ajoute 2 lignes (étire le tableau si besoin en tirant la poignée en bas à droite) : | id | titre | en_action | criticite | statut | |---|---|---|---|---| | PO-001 | Question expert normes | EXPERT | HAUTE | OUVERT | | PO-002 | Attente retour fournisseur | FOURNISSEUR | MOYENNE | OUVERT | | PO-003 | Accès outil obtenu | SE | BASSE | CLOS |

Renseigne aussi une date d'ouverture et une ligne de journal sur chaque point (texte libre). **Enregistre et ferme.**

Étape 6 — Synchronisation et vérification des KPI

```
python scripts/valider_un.py projet_TrainSystem/L09U1-TEST01-CLIM.xlsx
```

Attendu (console) : zéro erreur. Rouvre le fichier, onglet KPI :

KPI	Valeur attendue	Pourquoi
Avancement UO (%)	12,5	(50+100+0×10) ÷ 12 activités applicables
Heures consommées	30	10 + 20
Points ouverts	2	PO-001, PO-002
Points fermés	1	PO-003
Points critiques ouverts	1	PO-001 (HAUTE)
Dont balle chez fournisseur	1	PO-002
Dont balle chez expert	1	PO-001

KPI	Valeur attendue	Pourquoi
Reste à faire total (h)	210	formule Excel (recalculée à l'ouverture)
EAC	240	30 consommées + 210 RAF
Dérive à terminaison	0	EAC = heures vendues
Santé	ROUGE	un point critique est ouvert

L'onglet Dashboard doit refléter les mêmes valeurs (cartes).

i Les lignes « Reste à faire », « EAC », « Dérive », « Santé » sont des formules Excel : elles s'actualisent à l'ouverture du fichier, pas à la sync.

Étape 7 — Règles de qualité (VALIDATE)

1. Ouvre le fichier, mets l'avancement de la **ligne 2 à 150**. Enregistre, ferme.
2. Relance la sync.

Attendu : la sync se termine **en erreur**, avec un message explicite : [ERR] VALIDATE \$actifs.avancement : RANGE(0.0,100.0) : 1 valeur(s) hors plage ([150])

1. Remets **50**, enregistre, ferme, relance : **zéro erreur**, KPI inchangés (mêmes valeurs qu'à l'étape 6 — la sync est répétable).

Étape 8 — Le store (ce que verront les cockpits)

```
python -m src status --prefix uo.
```

Attendu : les 8 clés de l'UO avec leurs valeurs (avancement = 12.5, activites = [table: 12 lignes]...). C'est ce qui alimentera le cockpit ingénieur et le dashboard métier.

Étape 9 — Modifier le Manifeste : ajouter une instruction PUSH

Objectif : vérifier que modifier le `_Manifeste` suffit pour enrichir ce que l'UO publie dans le store — sans recréer le fichier.

Ouvre `L09U1-TEST01-CLIM.xlsx`, feuille `_Manifeste`. À la fin des instructions existantes, ajoute cette ligne :

```
PUSH $liv -> uo.L09U1-TEST01-CLIM.livrables_v2
```

(La variable `$liv` est déjà définie plus haut dans le Manifeste généré — c'est elle qui contient les livrables. On pousse vers une nouvelle clé `Livrables_v2` pour éviter tout conflit avec une éventuelle clé existante.)

Enregistre et ferme. Relance la sync :

```
python scripts/valider_un.py projet_TrainSystem/L09U1-TEST01-CLIM.xlsx
python -m src status --prefix uo.L09U1-TEST01-CLIM
```

Attendu : - La console affiche maintenant `PUSH = 9` (une de plus qu'avant). - `python -m src status` liste une nouvelle clé `uo.L09U1-TEST01-CLIM.livrables_v2` avec ses lignes. - Zéro erreur.

Ce que ça prouve : le Manifeste est la seule source de vérité — le modifier reconfigure le comportement immédiatement. Pas besoin de toucher au moteur.

Étape 10 — Ajouter une colonne à un tableau existant

Objectif : vérifier que le moteur absorbe sans erreur une colonne ajoutée par l'ingénieur, et qu'elle remonte dans le store.

Ouvre le fichier, feuille `Activites`. Ajoute une colonne `commentaire` à droite du tableau `tbl_act` (clique sur la cellule à droite du dernier en-tête, tape `commentaire` — Excel étend automatiquement le tableau). Renseigne un texte libre sur 2 ou 3 lignes. Enregistre et ferme.

```
python scripts/valider_un.py projet_TrainSystem/L09U1-TEST01-CLIM.xlsx
```

Puis vérifie les colonnes présentes dans le store avec ce snippet Python :

```
python -c "
import json, pathlib
store_path = pathlib.Path('output/store.json')
data = json.loads(store_path.read_text(encoding='utf-8'))
rows = data.get('uo.L09U1-TEST01-CLIM.activites', [])
if rows:
    print('Colonnes :', list(rows[0].keys()))
    print('Nb lignes :', len(rows))
else:
    print('Cle absente ou vide')
"
```

Attendu : - Zéro erreur. - La liste des colonnes inclut `commentaire` en plus des colonnes d'origine. - Les KPI sont inchangés (le moteur calcule sur `avancement`, pas sur `commentaire`).

Ce que ça prouve : les colonnes libres sont tolérées — un ingénieur peut enrichir son fichier sans casser la sync.

Étape 11 — Ajouter une feuille libre

Objectif : vérifier qu'une feuille non référencée dans le Manifeste est simplement ignorée.

Ouvre le fichier, crée une feuille nommée `Notes_perso` et tape quelques lignes de texte. Enregistre et ferme.

```
python scripts/valider_un.py projet_TrainSystem/L09U1-TEST01-CLIM.xlsx
```

Attendu : zéro erreur, comportement identique à avant. La feuille `Notes_perso` n'apparaît nulle part dans le store.

Étape 12 — Piège : renommer un tableau (comportement silencieux à connaître)

Objectif : comprendre ce qui se passe quand un tableau est renommé — pour ne pas être surpris si ça arrive en vrai.

Ouvre le fichier, feuille `OIL`. Dans le ruban **Création de tableau**, renomme le tableau `tbl_oil` en `tbl_oil_v2`. Enregistre et ferme.

```
python scripts/valider_un.py projet_TrainSystem/L09U1-TEST01-CLIM.xlsx
python -m src status --prefix uo.L09U1-TEST01-CLIM.oil
```

Attendu : - La console affiche **zéro erreur** — le moteur ne plante pas. - **PUSH reste à 9** : le PUSH s'exécute quand même, mais pousse une liste vide (le tableau `tbl_oil` est introuvable → `GET_TABLE` renvoie `[]`). - La clé `uo.L09U1-TEST01-CLIM.oil` reste dans le store mais contient **0 ligne** (`[]`). - Les KPI liés aux points ouverts tombent à **0** dans le Dashboard.

C'est la **dégradation gracieuse** : un tableau introuvable est traité comme vide, pas comme une erreur fatale. Utile pour la robustesse, trompeur pour le debug.

Remise en ordre : rouvre le fichier, renomme le tableau en `tbl_oil` (nom d'origine), enregistre, ferme, relance la sync. Les KPI reviennent.

Étape 13 — Modifier le FILE_ID (piège à éviter)

Objectif : comprendre pourquoi il ne faut pas changer le `FILE_ID` en cours de vie d'un fichier.

Ouvre le `_Manifeste`, change `FILE_ID: L09U1-TEST01-CLIM` en `FILE_ID: L09U1-TEST01-CLIM-V2`. Enregistre et ferme.

```
python scripts/valider_un.py projet_TrainSystem/L09U1-TEST01-CLIM.xlsx
```

Puis vérifie le contenu du store :

```
python -m src status --prefix uo.
```

Attendu : - Zéro erreur. - Le store contient maintenant **deux jeux de clés** : - `uo.L09U1-TEST01-CLIM.*` — l'ancienne identité (données périmées, non mises à jour) - `uo.L09U1-TEST01-CLIM-V2.*` — la nouvelle identité (données fraîches de cette sync) - Un cockpit qui consommerait les deux voit des doublons.

Remise en ordre : remets `FILE_ID: L09U1-TEST01-CLIM`, vide le store, relance.

```
python -m src store clear
python scripts/valider_un.py projet_TrainSystem/L09U1-TEST01-CLIM.xlsx
```

Étape 14 — Bilan libre (le plus important)

Mets-toi 10 minutes dans la peau d'un ingénieur qui reçoit ce fichier : crée une 2^e UO d'un autre type (ex. `L11U1-TEST02-FREIN`), remplis-la comme tu le ferais en vrai, et note **tout ce qui te gêne** : colonnes manquantes ou inutiles, libellés pas clairs, saisies pénibles, KPI faux ou absents, idées. Ces remarques valent plus que les OK/KO — elles feront la v2.

Récapitulatif des commandes

```
git pull
pip install -r requirements.txt
python projet_TrainSystem/parse_catalogue.py "CHEMIN\UO TS.docx" # étape 1 : catalogue json
python projet_TrainSystem/build_catalogue.py # étape 1 : catalogue xlsx
python projet_TrainSystem/creer_uo.py CODE --se "Nom" --heures N # étape 3 : créer une UO
python -m src store clear # remise à zéro du store
python scripts/valider_un.py projet_TrainSystem/CODE.xlsx # synchroniser un fichier
python -m src status # tout le store
python -m src status --prefix uo. # clés d'une UO seulement
```